

ELEC 374 Digital Systems Engineering
Laboratory Project

Winter 2026

Designing a Simple RISC Computer: Phase 3

1. Objectives

The objective of this project is to design, simulate, implement, and verify a simple RISC Computer (Mini SRC). So far, you have designed and functionally simulated the Datapath portion of the Mini SRC (except for *nop* and *halt* instructions that you will test in this phase). Phase 3 of this project consists of adding and testing the Control Unit in Mini SRC. You are to design the Control Unit in Verilog or VHDL. Testing will be done by Functional Simulation.

2. Preliminaries

2.1 Control Unit

A block diagram of the Control Unit for Mini SRC is shown in Figure 1. The Control Unit is at the heart of the processor. It accepts as input those signals that are needed to operate the processor and provides as output all control signals necessary to execute the instructions. The outputs from the Control Unit are the control signals that we have been using in previous phases to generate control sequences for the instructions of the Mini SRC.

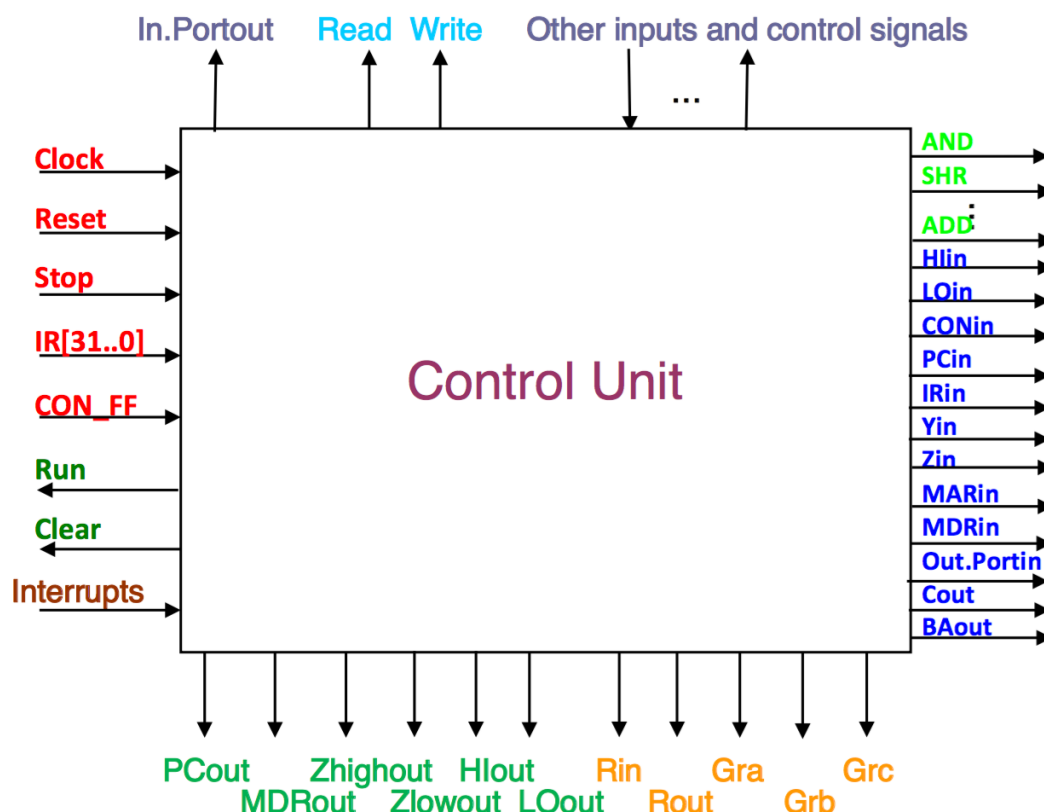


Figure 1: Block diagram of the Control Unit

The Control Unit generates the control signals from four principal sources:

- (a) The op-code fields of the IR
- (b) Signals from the Datapath such as CON FF, and the condition code registers (if any)
- (c) Control step information such as signals T0, T1, ...
- (d) External inputs such as Stop, Reset, Done (if memory is slow), and other signals such as interrupts (if any)

The external *Reset* input should get the Mini SRC to an initial state, where all registers including PC in the Datapath are set to 0, the *Run* indicator is set to 1, and the processor starts at Step T0. As the clock continues to run, instructions should be fetched and executed one after the other until a *halt* instruction is encountered, at which point the control stepping process should be halted and the *Run* indicator is set to 0. Note that, the external *Stop* input signal works the same way as the *halt* instruction.

During T0, T1, and T2, the control signals that are asserted to implement the “instruction fetch” sequence are independent of the bits in the Instruction Register. For instance, in Step T0, the control signals PCout, MARin, IncPC and Zin are set to 1. In Step T1, the control signals Zlowout, PCin, Read, and MDRin are set to 1. In Step T2, the control signals MDRout and IRin are set to 1. However, from Step T3 onward, until the current instruction is completed, the control signals that are asserted are a function of both Step Ti and the op-code bits in the IR register.

In the following, you will see two different methods to design your Control Unit. There is a trade-off between the two methods. Method 1 is clearly the easier method, but it may generate more hardware. You are, of course, welcome to come up with your own design.

Method 1: It is possible to write the Verilog/VHDL code without worrying about the combinational logic expressions for each control signal. Therefore, the code will come clean, and the instructions will be executed in the most efficient manner. However, this approach may generate more hardware. The following sample Verilog/VHDL code is provided as a starting point for this method, which you may need to verify and revise for your Control Unit:

// this is the Verilog sample code for Method 1 for the Control Unit

```
`timescale 1ns/10ps
module control_unit (
    output reg    Gra, Grb, Grc, Rin, ..., Rout,          // define the inputs and outputs to your Control Unit here
                  Yin, Zin, PCout, IncPC, ..., MARin,
                  Read, Write, ..., Clear,
                  ADD, AND, ..., SHR,
    input         [31:0] IR,
    input         Clock, Reset, Stop, ..., Con_FF);
    parameter     reset_state = 4'b0000, fetch0 = 4'b0001, fetch1 = 4'b0010, fetch2 = 4'b0011,
                  add3 = 4'b0100, add4 = 4'b0101, add5 = 4'b0110, ...;
    reg           [3:0] present_state = reset_state; // adjust the bit pattern based on the number of states

always @(posedge Clock, posedge Reset) // finite state machine; if clock or reset rising-edge
begin
    if (Reset == 1'b1) present_state = reset_state;
    else case (present_state)
        reset_state:    present_state = fetch0;
        fetch0:         present_state = fetch1;
        fetch1:         present_state = fetch2;
        fetch2:         begin
```

```

                                case (IR[31:27])    // inst. decoding based on the opcode to set the next state
                                    5'b00000: present_state = add3;    // this is the add instruction
                                    ⋮
                                endcase
                                end
                                add3: present_state = add4;
                                add4: present_state = add5;
                                ⋮
                                endcase
                                end

always @(present_state)    // do the job for each state
begin
    case (present_state)    // assert the required signals in each state
        reset_state: begin
            Gra <= 0; Grb <= 0; Grc <= 0; Yin <= 0;    // initialize the signals
            ⋮
        end
        fetch0: begin
            PCout <= 1;    // see if you need to de-assert these signals
            MARin <= 1;
            IncPC <= 1;
            Zin <= 1;
        end
        add3: begin
            Grb <= 1; Rout <= 1;
            Yin <= 1;
        end
        ⋮
    endcase
end
endmodule

```

-- this is the VHDL sample code for Method 1 for the Control Unit

library ieee;

use ieee.std_logic_1164.all;

entity control_unit is -- define the inputs and outputs to your Control Unit here

```

    port (Clock, Reset, Stop, ..., CON_FF:      in    std_logic;
          IR:                                     in    std_logic_vector(31 downto 0);
          Gra, Grb, Grc, Rin, ..., Rout:         out   std_logic;
          Yin, Zin, PCout, IncPC, ..., MARin:     out   std_logic;
          Read, Write, ..., Clear:               out   std_logic;
          ADD, AND, ..., SHR:                    out   std_logic);

```

end control_unit;

architecture behavior of control_unit is

type state is (reset_state, fetch0, fetch1, fetch2, add3, add4, add5, ...);

signal present_state: state;

begin

```

    process (Clock, Reset) -- finite state machine

```

```

begin
  if (Reset = '1') then          -- reset the processor
    present_state <= reset_state;
  elsif (Clock'event and Clock = '1') then  -- if clock rising-edge
    case present_state is
      when reset_state =>
        present_state <= fetch0;
      when fetch0 =>
        present_state <= fetch1;
      when fetch1 =>
        present_state <= fetch2;
      when fetch2 =>  -- instruction decoding based on the opcode to set the next state
        case IR(31 downto 27) is
          when "00000" =>  -- this is the add instruction
            present_state <= add3;
            ⋮
          when others =>
            end case;
        when add3 =>
          present_state <= add4;
        when add4 =>
          present_state <= add5;
          ⋮
        when others =>
          end case;
      end case;
    end if;
  end process;

  process (present_state)        -- do the job for each state
  begin
    case present_state is        -- assert the required signals in each state
      when reset_state =>
        Gra <= '0';              -- initialize the signals
        Grb <= '0';
        Grc <= '0';
        Yin <= '0';
        ⋮
      when fetch0 =>
        PCout <= '1';  -- see if you need to de-assert these signals
        MARin <= '1';
        IncPC <= '1';
        Zin <= '1';
      when add3 =>
        Grb <= '1';
        Rout <= '1';
        Yin <= '1';
        ⋮
      when nop =>
      when others =>
        end case;
    end process;
  end behavior;

```

Method 2: In this approach, you will need to derive all control signal setting conditions for all instructions. For this, you must examine all of the control sequences of the machine. The logic for each control signal is generated by going through the control sequences looking for every occurrence of that control signal and writing the Boolean equation for the signal. For example, the Zlowout signal occurs at T1 in all instructions, at T5 in *and*, *or*, *add*, *sub*, *mul*, *div*, *shr*, *shra*, *shl*, *ror*, *rol*, *ld*, and *ldi* instructions, and in some T states for other instructions. Therefore,

$$\text{Zlowout} = \text{T1} + \text{T5} \cdot (\text{AND} + \text{OR} + \text{ADD} + \text{SUB} + \text{MUL} + \text{DIV} + \text{SHR} + \text{SHRA} + \text{SHL} + \text{ROR} + \text{ROL} + \dots) + \dots$$

The problem with this approach is that the logic for each control signal may change with the addition of new instructions, therefore this approach is provided here merely for the sake of completeness. You are advised to use Method 1, especially if you intend to extend the scope of the project by adding new instructions, etc.

The following sample Verilog/VHDL code is provided as a starting point for Method 2, which you may need to verify and revise for your Control Unit:

```
// this is the Verilog sample code for Method 2 for the Control Unit
`timescale 1ns/10ps
module control_unit (
    output reg    Gra, Grb, Grc, Rin, ..., Rout,    // define the inputs and outputs to your Control Unit here
                Yin, Zin, PCout, IncPC, Zlowout, ..., MARin,
                Read, Write, ..., Clear,
                ADD, AND, ..., SHR,
    input         [31:0] IR,
    input         Clock, Reset, Stop, ..., Con_FF);

parameter       reset_state = 4'b0000, S0 = 4'b0001, S1 = 4'b0010, ...;
reg             [3:0] present_state = reset_state;    // adjust the bit pattern based on the number of states
reg             T0, T1, T2, T3, T4, T5, ..., ;
reg             ADD_s, SUB_s, AND_s, OR_s, ..., ;

always @(posedge Clock, posedge Reset, ...)    // finite state machine; if clock or reset rising-edge
begin
    if (Reset == 1'b1) present_state = reset_state;
    else begin
        T0 <= 0; T1 <= 0; T2 <= 0; T3 <= 0; T4 <= 0; T5 <= 0;
        case (present_state)
            reset_state: begin
                present_state = S0;
                T0 <= 1;
            end
            S0: begin
                present_state = S1;
                T1 <= 1;
            end
            ...
        endcase
    end
end
end
```

```

always @(IR)
begin
    ADD_s <= 0; AND_s <= 0; ...
    case (IR[31:27])
        5'b00000:    ADD_s <= 1;    // inst. decoding based on the opcode
        5'b00010:    AND_s <= 1;    // this is the add instruction
        5'b00010:    AND_s <= 1;    // this is the and instruction
        ...
    endcase
end

```

```

always @(Clock, T0, T1, ...)
begin
    ADD <= ADD_s && T4;                // control signal assignment
    Zlowout <= T1 || (T5 && (AND_s || OR_s || ADD_s || SUB_s || ...)) || ...;
    ...
end
endmodule

```

-- this is the VHDL sample code for Method 2 for the Control Unit

```

library ieee;
use ieee.std_logic_1164.all;

entity control_unit is
    -- define the inputs and outputs to your Control Unit here
    port(Clock, Reset, Stop, ..., CON_FF: in std_logic;
          IR: in std_logic_vector(31 downto 0);
          Gra, Grb, Grc, Rin, ..., Rout: out std_logic;
          Yin, Zin, PCout, IncPC, Zlowout, ..., MARin: out std_logic;
          Read, Write, ..., Clear: out std_logic;
          ADD, AND, ..., SHR: out std_logic);
end control_unit;

```

```

architecture behavior of control_unit is
    signal T0, T1, T2, T3, T4, T5, ... : std_logic;
    signal ADD_s, SUB_s, AND_s, OR_s, ... : std_logic;
    type state is (reset_state, S0, S1, ...);
    signal present_state: state;
begin
    process (Clock, Reset, ...) -- finite state machine
    begin
        if (Reset = '1') then
            present_state <= reset_state;
        elsif (Clock'event and Clock = '1') then -- if clock rising-edge
            T0 <= '0'; T1 <= '0'; T2 <= '0'; T3 <= '0'; T4 <= '0'; T5 <= '0'; ...
            case present_state is
                when reset_state =>
                    present_state <= S0;
                    T0 <= '1';
                when S0 =>
                    present_state <= S1;
                    T1 <= '1';
                ...
            end case
        end if
    end process
end

```

```

        when others =>
            end case;
        end if;
    end process;

    process (IR)
    begin
        ADD_s <= '0'; AND_s <= '0'; ...
        case IR(31 downto 27) is -- inst. decoding based on the opcode
            when "00000" =>
                ADD_s <= '1'; -- this is the add instruction
            when "00010" =>
                AND_s <= '1'; -- this is the and instruction
            :
            when others =>
                end case;
        end process;

    process (Clock, T0, T1, ...)
    begin
        ADD <= ADD_s AND T4; -- control signal assignment
        Zlowout <= T1 OR (T5 AND (AND_s OR OR_s OR ADD_s OR SUB_s OR ...)) OR ...;
        :
    end process;
end behavior;

```

3. Procedure

3.1) Use one of the above methods, or come up with your own design style, and write your Verilog/VHDL code to implement the Control Unit and add to your Datapath.

3.2) Run a functional simulation of the following program on Mini SRC and demonstrate it to one of the TAs. This program is provided to test the control unit and instructions in Mini SRC, except for brnz/brzr Branch and input/output instructions that will be included in the test code for Phase 4.

Encode your program in memory with the starting address zero. Initialize registers R0 - R15 and the PC to 0 with the *Reset* input signal. Initialize memory locations 0x89 and 0xA3 with the 32-bit hexadecimal values 0xA7 and 0x68, respectively.

Minimum outputs are IR, PC, MDR, MAR, R0 – R15, HI, and LO. Add any other signals you would like to observe to convince yourself and the TA that your design works fine.

Address	ORG	0	
0	ldi	R5, 0x43	; R5 = 0x43
	ldi	R5, 6(R5)	; R5 = 0x49
	ld	R4, 0x89	; R4 = (0x89) = 0xA7
	ldi	R4, 4(R4)	; R4 = 0xAB
	ld	R0, -8(R4)	; R0 = (0xA3) = 0x68
	ldi	R2, 4	; R2 = 4
	ldi	R5, 0x87	; R5 = 0x87
	brmi	R5, 3	; continue with the next instruction (will not branch)

	ldi	R5, 5(R5)	; R5 = 0x8C
	ld	R1, -3(R5)	; R1 = (0x8C - 3) = (0x89) = 0xA7
	nop		
	brpl	R1, 2	; continue with the instruction at "target" (will branch)
	ldi	R3, 7(R5)	; this instruction will not execute
	ldi	R7, -4(R3)	; this instruction will not execute
target:	add	R7, R5, R2	; R7 = 0x90
	addi	R1, R1, 3	; R1 = 0xAA
	neg	R1, R1	; R1 = 0xFFFFF56
	not	R1, R1	; R1 = 0xA9
	andi	R1, R1, 0xF	; R1 = 9
	ror	R4, R0, R2	; R4 = 0x80000006
	ori	R1, R4, 5	; R1 = 0x80000007
	shra	R4, R1, R2	; R4 = 0xF8000000
	shr	R5, R5, R2	; R5 = 0x8
	st	0xA3, R5	; (0xA3) = 0x8 new value in memory with address 0xA3
	rol	R5, R0, R2	; R5 = 0x680
	or	R7, R2, R0	; R7 = 0x6C
	and	R4, R5, R0	; R4 = 0
	st	0x89(R4), R7	; (0x89) = 0x6C new value in memory with address 0x89
	sub	R0, R5, R7	; R0 = 0x614
	shl	R4, R5, R2	; R4 = 0x6800
	ldi	R7, 7	; R7 = 7
	ldi	R3, 0x19	; R3 = 0x19
	mul	R3, R7	; HI = 0; LO = 0xAF
	mfhi	R1	; R1 = 0
	mflo	R6	; R6 = 0xAF
	div	R3, R7	; HI = 4, LO = 3
	ldi	R8, 2(R7)	; R8 = 9 setting up argument registers
	ldi	R9, -4(R3)	; R9 = 0x15 R8, R9, R10, and R11
	ldi	R10, 3(R6)	; R10 = 0xB2
	ldi	R11, 5(R1)	; R11 = 5
	jal	R10	; subA address 0xB2 in R10 into PC; return address 0x29 into R12
0x29	halt		; upon return, the program halts
subA:	ORG	0xB2	; procedure subA
0xB2	add	R14, R8, R10	; R13 and R14 are return value registers
	sub	R13, R9, R11	; R14 = 0xBB, R13 = 0x10
	sub	R14, R14, R13	; R14 = 0xAB
	jr	R12	; return from subA procedure with address 0x29 in R12

4. Report

Upload your Phase 3 Lab report (one per group) in PDF format to onQ by 11:59pm on the day of your Phase 3 demo. Phase 3 report consists of:

- Your Verilog/VHDL code (and schematic, if any)
- Functional simulation run of the program
- Printouts of the contents of memory before and after the program run